

Back End/Devops Engineer

Technical Vetting Assignment

Instructions & Requirements

Take Home Assignment: Fleet Health “Digital Twin” Service

Context

Your team runs 20+ Ubuntu servers hosting Node.js applications in Docker. You need a lightweight internal “digital twin” service to record and report server health signals and basic service status. The goal is to build a small Node.js service that ingests heartbeat data from hosts, persists it to a SQL database, and exposes simple reporting. It should be containerized for local development and include basic operational scripts that reflect day to day work in a Linux environment.

Requirements

Build a small, production minded system that you can run locally. Focus on practicality, observability, and the kinds of workflows you’d use when supporting running services.

Constraints and core stack:

- Node.js (JavaScript or TypeScript) for the service (Node 18+; prefer a current LTS)
- PostgreSQL for SQL storage
- Docker and Docker Compose for local orchestration
- Bash for operational scripting (Linux/Ubuntu oriented)

Example heartbeat payloads your system will ingest:

- Host heartbeat (sent periodically)
 - ```
{
 "host": "api-01",
 "timestamp": "2025-03-10T15:04:05Z",
 "cpu_load": 0.42,
 "mem_used_mb": 512,
 "services": [
 {"name": "api", "healthy": true},
 {"name": "worker", "healthy": false}
],
 "ip": "10.0.0.21"
}
```
- Optional error/incident signal (if you choose to support it)
  - ```
{
  "host": "api-01",
  "timestamp": "2025-03-10T15:05:00Z",
  "type": "error",
  "message": "upstream timeout contacting db"
}
```

Design decisions (API shape, schema, error handling, logging format) are up to you, within the tasks below.

Task 1: Backend service for ingest and reporting

Build a Node.js HTTP service that:

- Accepts heartbeat JSON and persists it to PostgreSQL.
- Exposes endpoints to retrieve:
 - The latest status for a given host.
 -

A summary across all known hosts (e.g., latest heartbeat per host with a simple healthy/unhealthy indicator derived from the most recent data).

- Uses SQL queries (not only in memory state) for all reads.
- Uses environment variables for configuration (e.g., DB connection, port).
- Uses parameterized queries and a reusable database connection strategy.
- Emits structured logs suitable for production (e.g., one line JSON per event or another consistent structure).
- Shuts down cleanly on termination signals by closing the HTTP server and database resources.

Notes:

- You may use any minimal HTTP framework you prefer (or none).
- The data model is yours to design. Optimizing for “latest per host” queries is expected.

Task 2: Containerization and local orchestration

Containerize the service and run it locally with PostgreSQL using Docker Compose.

- Provide a multi stage Dockerfile for the app image and ensure it does not run as root.
- Add a container health check for the app image.
- Compose file should:
 - Run the app and a PostgreSQL service.
 - Provide a persistent named volume for PostgreSQL data.
 - Configure the DB via environment variables (database, user, password).
 - Include a database health check and only start the app when the DB is healthy.
- Include a simple way to initialize database schema (migration or bootstrap step of your choice).

Task 3: Ops and diagnostics script (Bash)

Provide a single Bash script (e.g., ops.sh) to streamline common operations:

- Local lifecycle: start, stop, restart, status of the stack (app + DB).
- Logs: tail aggregated logs for app and DB with a simple filter option (e.g., by host).
- Seed/simulate: send a small number of synthetic heartbeats for multiple hosts to exercise the system.
- Snapshot: create a local database backup file.
- Optional remote helper: a subcommand that, given SSH parameters via environment variables, attempts to run a simple diagnostic command remotely (e.g., print “docker ps” output). This should be safe to run without real credentials present.

Keep the script POSIX ish where practical, and assume a Linux shell environment.

Deliverables

1. A single GitHub repository containing:

- All the source code for the assignment
- A README.md with instructions to run locally.
- A SOLUTION.md describing the design and trade-offs you made.

2. Video Demo (5-10 minutes using Loom or similar):

- Demonstrate the working project by doing a voice over
- Walk through your code architecture
- Explain key design decisions and any trade-offs made
- Highlight interesting technical implementations

Time Estimate

This should take ± 4 hours to complete, but you can take as long as you need.